UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
THÀNH PHỐ HỒ CHÍ MINH

HP:012012710404

Machine Learning: Regression algorithms

Linear models: Linear Regression, Ridge Regression, Lasso Regression, ElasticNet Regression

Group: 3
Nguyễn Xuân Đông
Mai Hoàng Danh
Võ Khôi Nguyên
Võ Thanh Phú
Trương Thành Công
Nguyễn Minh Nhật
Mai An Thịnh

GVHD: TS. Nguyễn Thị Khánh Tiên

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

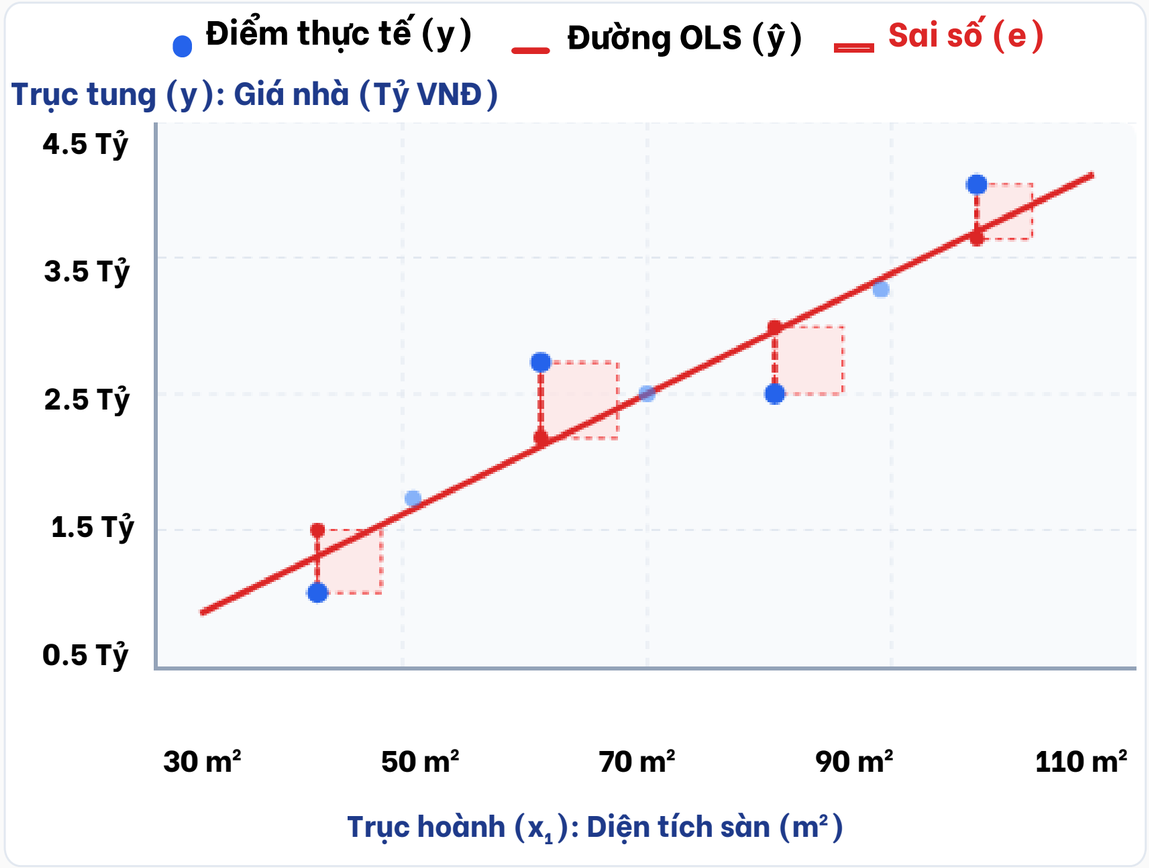
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Hồi Quy Tuyến Tính (Linear Regression)

Bản Chất Linear Regression

Học có giám sát dự đoán biến mục tiêu liên tục. Đầu ra biến thiên tuyến tính theo các biến đặc trưng đầu vào.

$$\hat{y}(w, x) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_p x_p$$



1. Phương pháp bình phương tối thiểu (OLS)

$$\mathrm{SSR} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

2. Giải tích (Normal Equation)

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

3. Gradient Descent

Khởi tạo w ngẫu nhiên, cập nhật ở từng bước đến khi w tối ưu

Triển khai thuật toán

1. Khởi tạo (__init__)

Tạo biến lưu trọng số (w) và hệ số chặn (b).

2. Huấn luyện (fit)

Thêm cột 1 vào ma trận để tính hệ số chặn. Dùng Phương trình chuẩn (Normal Equation) để tìm ra bộ trọng số tối ưu nhất trong 1 bước tính.

3. Dự đoán (predict)

Dự đoán giá trị theo công thức:

$$\hat{y} = X \cdot w + b$$

4. Đánh giá (score)

Tự tính độ chính xác R bình để biết mô hình giải thích được bao nhiêu % dữ liệu.

```python
class LinearRegressionScratch:
    def __init__(self):
        self.coef_ = None
        self.intercept_ = None

    def fit(self, X, y):
        # 1. Thêm cột 1 cho bias: X_b = [1, X]
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        # 2. Nghiệm đóng Normal Equation qua Pseudo-Inverse
        theta = np.linalg.pinv(X_b) @ y
        self.intercept_, self.coef_ = theta[0], theta[1:]
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_

    def score(self, X, y):
        # Đánh giá bằng hệ số xác định R2
        ss_res = np.sum((y - self.predict(X)) ** 2)
        ss_tot = np.sum((y - np.mean(y)) ** 2)
        return 1.0 - (ss_res / ss_tot)
```

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Bài toán thực tế & Ưu Nhược điểm

- **Bộ dữ liệu**: housing_market_data.csv (Khảo sát 85 căn nhà, 17 đặc trưng: số phòng, thuế, ô nhiễm, tỷ lệ dân nghèo...).
- **Biến mục tiêu** (y): Giá nhà (house_price - nghìn USD).

Kết quả mô hình

Tiêu chí đánh giá	Linear Regression
R2 Train	0.92 (92%)
R2 Test	**0.59 (59%)**
RMSE	5.09 (nghìn USD)
Kết luận	Chênh lệch Train-Test 33% (Rất xa)

Ưu điểm

- Tốc độ: Tính toán tức thì, không cần vòng lặp tốn tài nguyên.
- Dễ giải thích: Nhìn vào trọng số biết ngay biến nào làm tăng/giảm giá nhà.

Nhược điểm

- Bị đa cộng tuyến: Khi các biến dính líu đến nhau (thuế, số phòng, vị trí), mô hình bị nhiễu loạn trọng số.
- Không tự lọc biến: Cả 17 biến đều bị ép giữ lại (0/17 biến bị loại), kể cả các biến gây nhiễu.

Mô hình bị Overfitting rất nặng:
Học thuộc lòng dữ liệu Train nhưng trên dữ liệu mới (Test) bị sụt giảm nghiêm trọng (từ 92% rớt xuống 59%).

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Ridge Regression

Ridge Regression là một biến thể của hồi quy tuyến tính, sử dụng kỹ thuật L2 Regularization để hạn chế hiện tượng quá khớp (Overfitting) và cải thiện độ ổn định của mô hình, đặc biệt khi dữ liệu có hiện tượng đa cộng tuyến (Multicollinearity) hoặc số lượng biến độc lập lớn hơn số lượng quan sát.

Hàm mục tiêu của Ridge Regression

$$J(w) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2 + \alpha\sum_{j=1}^{p} w_j^2$$

Mục Tiêu của Ridge Regression

- Cải thiện tính ổn định và khả năng tổng quát hóa (generalization) của mô hình hồi quy tuyến tính.
- Thêm một điều kiện phạt (penalty term) vào hàm mất mát.
- Giảm thiểu độ lớn của các hệ số hồi quy
- Tìm bộ hệ số trọng số w cho các biến độc lập sao cho hàm mục tiêu (dựa trên hàm mất mát bình phương trung bình - MSE) đạt giá trị tối ưu (nhỏ nhất).

Triển khai thuật toán

1. Khởi tạo (__init__)

Tạo siêu tham số α (độ phạt)

α càng lớn → kìm hãm trọng số càng mạnh để chống học vẹt (Overfitting).

2. Huấn luyện (fit)

Thêm cột 1 cho hệ số chặn.

Tạo ma trận đơn vị I để không phạt hệ số chặn

Tìm Nghiệm đóng Ridge trong 1 bước

3. Dự đoán (predict)

Dự đoán giá trị theo công thức:

$$\hat{y} = X \cdot w + b$$

```python
class RidgeRegressionScratch:
    def __init__(self, alpha=10.0):
        self.alpha = float(alpha)
        self.coef_, self.intercept_ = None, None

    def fit(self, X, y):
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        # Tạo ma trận đơn vị I (I[0,0] = 0 để không phạt bias)
        I = np.eye(X.shape[1] + 1)
        I[0, 0] = 0.0
        # Nghiệm đóng Ridge: (X^T X + alpha * I)^(-1) X^T y
        theta = np.linalg.solve(X_b.T @ X_b + self.alpha * I, X_b.T @ y)
        self.intercept_, self.coef_ = theta[0], theta[1:]
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_

    def score(self, X, y):
        return 1.0 - np.sum((y - self.predict(X)) ** 2) / np.sum((y - np.mean(y)) ** 2)
```

4. Đánh giá (score)

Tự tính hệ số xác định R bình phương để đo độ chuẩn xác.

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Bài toán thực tế & Ưu Nhược điểm

Mục tiêu: Áp dụng Ridge lên cùng tập dữ liệu bất động sản để kiểm chứng khả năng kiểm soát Overfitting.

Kết quả mô hình

Tiêu chí đánh giá	Linear Regression	Ridge Regression
R2 Train	0.92 (92%)	0.81 (81%)
R2 Test	**0.59 (59%)**	**0.72 (72%)**
RMSE	5.09 (nghìn USD)	4.29 (nghìn USD)
Kết luận	Lệch Train-Test 33%	Lệch Train-Test 9%

Ưu điểm

- Tốc độ: Tính toán nhanh
- Giảm Đa cộng tuyến: tính toán ổn định không còn bị loạn trọng số dù các biến có dính líu chặt chẽ với nhau (thuế, diện tích, vị trí).
- Kiểm soát Overfitting cực tốt: Kìm hãm hệ số không bị bùng nổ, giúp R2 Test tăng vọt từ 59%→72%

Nhược điểm

- Không loại bỏ được biến thừa: Chỉ thu nhỏ trọng số chứ không triệt tiêu về 0, buộc người dùng thực tế vẫn phải thu thập đủ cả 17 biến (kể cả biến gây nhiễu).
- Tăng độ chệch (Bias): Nếu chọn hệ số phạt α quá lớn, mô hình sẽ bị cứng nhắc và mất độ chính xác.

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

LASSO REGRESSION

LASSO (Least Absolute Shrinkage and Selection Operator) là kỹ thuật điều chỉnh được sử dụng để ngăn ngừa hiện tượng quá khớp (overfitting) trong các mô hình hồi quy tuyến tính

Hàm mục tiêu của Lasso Regression

$$J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \alpha\sum_{j=1}^{p}|w_j|$$

Mục tiêu của Lasso Regression

- Giảm sai số dự đoán.
- Hạn chế hiện tượng quá khớp (Overfitting).
- Thu nhỏ các hệ số hồi quy.
- Tự động lựa chọn đặc trưng (Feature Selection) bằng cách đưa một số hệ số về đúng 0.

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Triển khai thuật toán

1. Khởi tạo (__init__)

Tạo siêu tham số α (độ phạt)

2. Toán tử cắt mềm (_soft_thresholding)

Nếu biến có tác động yếu hơn ngưỡng phạt thì **triệt tiêu trọng số về đúng bằng 0.**

3. Huấn luyện hạ tọa độ (fit)

Dùng thuật toán Coordinate Descent: Tối ưu từng trọng số w một cách tuần tự cho đến khi sai số đạt mức tối thiểu.

4. Dự đoán (predict)

Dự đoán giá trị: $\hat{y} = X \cdot w + b$

5. Đánh giá (score)

Tự tính hệ số xác định R2 để đo lường độ chính xác của mô hình.

```python
class LassoRegressionScratch:
    def __init__(self, alpha=1.05, max_iter=2000, tol=1e-5):
        self.alpha, self.max_iter, self.tol = alpha, max_iter, tol

    def _soft_thresholding(self, rho, threshold):
        if rho > threshold:
            return rho - threshold
        if rho < -threshold:
            return rho + threshold
        return 0.0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.intercept_ = float(np.mean(y))
        self.coef_ = np.zeros(n_features)
        r = (y - self.intercept_).copy()
        norm_x = np.sum(X**2, axis=0)

        for _ in range(self.max_iter):
            for j in range(n_features):
                rho_j = np.dot(X[:, j], r) + norm_x[j] * self.coef_[j]
                w_new = (
                    self._soft_thresholding(rho_j, n_samples * self.alpha) / norm_x[j]
                )
                r += X[:, j] * (self.coef_[j] - w_new)
                self.coef_[j] = w_new
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_

    def score(self, X, y):
        return 1.0 - np.sum((y - self.predict(X)) ** 2) / np.sum((y - np.mean(y)) ** 2)
```

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Bài toán thực tế & Ưu Nhược điểm

Mục tiêu: Áp dụng Lasso nhằm chọn lọc đặc trưng—triệt tiêu các biến dư thừa mà vẫn duy trì độ chính xác cao.

Kết quả mô hình

Tiêu chí đánh giá	Linear (OLS)	Ridge Regression	Lasso Regression
R2 Train	0.92 (92%)	0.81 (81%)	**0.74 (74%)**
R2 Test	0.59 (59%)	0.72 (72%)	**0.67 (67%)**
RMSE	5.09 (nghìn USD)	4.22 (nghìn USD)	**4.58 (nghìn USD)**
Số biến bị loại	0 / 17 biến	0 / 17 biến	**11 / 17 biến**

Ưu điểm

- Lọc biến siêu gọn (11/17 biến bị loại): Triệt tiêu 11 biến nhiễu về đúng bằng 0, chỉ giữ lại 6 biến quan trọng nhất quyết định giá nhà.
- Mô hình thưa, kiểm soát Overfitting: Độ lệch Train-Test thu hẹp chỉ còn 7% (Test đạt 67%), mô hình cực kỳ gọn nhẹ, dễ giải thích

Nhược điểm

- "Chọn 1 bỏ 1" thiếu ổn định: Khi gặp nhóm biến tương quan cao (như số phòng & diện tích), Lasso chỉ chọn bừa 1 biến đại diện và xóa bỏ biến còn lại.
- Độ chính xác bị giảm: Do phạt quá gắt nên Lasso vô tình loại bớt một phần thông tin hữu ích.

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

ElasticNet Regression

Vì sao cần ElasticNet thay vì Lasso?

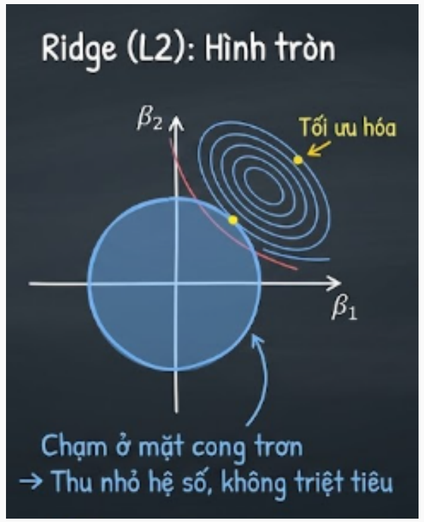
- Khái niệm: Là mô hình kết hợp đồng thời chuẩn phạt L1 (Lasso) và L2 (Ridge).
- Giải quyết điểm yếu của Lasso: Khi các biến có tính đa cộng tuyến nhóm, Lasso thường chọn bừa 1 biến và vứt bỏ các biến còn lại. ElasticNet kích hoạt Hiệu ứng gom nhóm của Ridge để cùng giữ lại các biến quan trọng.

Hàm mục tiêu của Lasso Regression

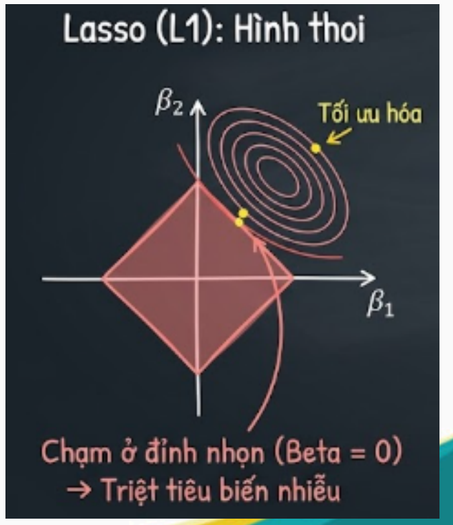
$$J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \alpha\left[\rho\sum_{j=1}^{p}|w_j| + \frac{1-\rho}{2}\sum_{j=1}^{p}w_j^2\right]$$

Bản chất hình học của vùng ràng buộc ElasticNet

- Lasso: Hình thoi (có đỉnh nhọn → lọc biến về 0).
- Ridge: Hình tròn (cạnh cong → co nhỏ trọng số, không triệt tiêu).
- ElasticNet: Hình thoi bo góc (vừa có đỉnh nhọn để lọc biến, vừa có cạnh cong để gom nhóm biến tương quan).







UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Triển khai thuật toán

1. Khởi tạo (__init__)

Tạo 2 siêu tham số: α (độ phạt) và l1_ratio (tỷ lệ giữa L1 và L2L2), như ở đây là 0.3 tương đương 30% Lasso + 70% Ridge)

2. Cơ chế phạt kép (L1+L2)

Tử số dùng toán tử cắt mềm (L1) để lọc biến rác. Mẫu số cộng thêm L2 để giữ lại nhóm biến tương quan.

3. Huấn luyện hạ tọa độ (fit)

Dùng thuật toán Coordinate Descent: Tối ưu từng trọng số w kết hợp đồng thời cả 2 mức phạt L1 và L2.

4. Dự đoán & Đánh giá (predict & score)

Dự đoán giá trị: $\hat{y} = X \cdot w + b$

Tự tính hệ số xác định R2 để đo lường độ chính xác của mô hình.

```python
class ElasticNetRegressionScratch:
    def __init__(self, alpha=0.53, l1_ratio=0.30, max_iter=2000, tol=1e-5):
        self.alpha, self.l1_ratio = alpha, l1_ratio
        self.max_iter, self.tol = max_iter, tol

    def _soft_thresholding(self, rho, threshold):
        if rho > threshold:
            return rho - threshold
        if rho < -threshold:
            return rho + threshold
        return 0.0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.intercept_ = float(np.mean(y))
        self.coef_ = np.zeros(n_features)
        r = (y - self.intercept_).copy()
        norm_x = np.sum(X**2, axis=0)
        l1_pen = n_samples * self.alpha * self.l1_ratio
        l2_pen = n_samples * self.alpha * (1.0 - self.l1_ratio)

        for _ in range(self.max_iter):
            for j in range(n_features):
                rho_j = np.dot(X[:, j], r) + norm_x[j] * self.coef_[j]
                w_new = self._soft_thresholding(rho_j, l1_pen) / (norm_x[j] + l2_pen)
                r += X[:, j] * (self.coef_[j] - w_new)
                self.coef_[j] = w_new
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_

    def score(self, X, y):
        return 1.0 - np.sum((y - self.predict(X)) ** 2) / np.sum((y - np.mean(y)) ** 2)
```

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

Bài toán thực tế & Ưu Nhược điểm

- **Mục tiêu**: Mục tiêu: Áp dụng ElasticNet nhằm kết hợp cả Ridge và Lasso, tối ưu hóa toàn diện bài toán giá nhà.

Kết quả mô hình

Tiêu chí đánh giá	1. Linear (OLS)	2. Ridge Regression	3. Lasso Regression	4. ElasticNet
R2 Train	**0.92 (92%)**	0.81 (81%)	0.74 (74%)	0.77 (77%)
R2 Test	0.59 (59%)	**0.72 (72%)**	**0.67 (67%)**	**0.70 (70%)**
RMSE	5.09 (nghìn USD)	4.22 (nghìn USD)	4.58 (nghìn USD)	**4.38 (nghìn USD)**
Số biến bị loại	0 / 17	0 / 17	**11 / 17 biến**	**4 / 17 biến**
Kết luận	**Overfitting nặng** (Lệch Train-Test 33%)	**Kìm hãm Overfitting đỉnh nhất** (Lệch chỉ 9%)	**Lọc biến rác mạnh nhất**, tiết kiệm chi phí	**Cân bằng** giữa độ chính xác & độ gọn

Ưu điểm

- Kết hợp (L1+L2): Khắc phục điểm yếu của Lasso nhờ hiệu ứng gom nhóm — giữ lại trọn vẹn các biến tương quan quan trọng (vị trí, số phòng) thay vì chọn bừa 1 biến.
- Cân bằng tối ưu toàn diện: Vừa lọc bỏ được 4/17 biến nhiễu, vừa giữ được độ chính xác rất cao (R2 Test đạt 70%, cao hơn hẳn Lasso 67%, độ lệch chỉ 7%).

Nhược điểm

- Cấu hình phức tạp, tốn thời gian: Phải dò tìm cả 2 siêu tham số (α & l1_ratio) thay vì chỉ 1 tham số như Ridge hay Lasso.
- Hiện tượng co kép: Bị cả 2 chuẩn phạt cùng tác động nên có thể làm tăng nhẹ độ chệch (Bias) ở một số trọng số quan trọng.


Thank
you